QUALITY ASSURANCE & SOFTWARE VERIFICATION

Comprehensive Testing Architecture & Verification Report

Infrastructure Project Delay & Cost Overrun Intelligence Agent

TEST FRAMEWORK	TEST SUITE STATUS	EXECUTION TIME	INTEGRATION SCOPE
Pytest 9.1.1	15 / 15 Passed (100%)	2.90 Seconds	12,633 Real Activities

1. Executive Summary & Testing Philosophy

In mission-critical infrastructure construction management, inaccurate schedule or financial projections can trigger multi-crore contractual claims, unjustified vendor penalties, or flawed resource allocation. Testing this AI decision-support platform required an unwavering architectural principle:

> **Core Testing Doctrine**
>
> **Deterministic arithmetic must be mathematically provable; non-deterministic AI must be evidence-grounded and anti-hallucinatory.** Mathematical calculations (CPM float, overdue days, cost variance, stock deficits, productivity rates) are tested against hand-calculated ground truths. AI and RAG components are tested to ensure they cite exact verbatim contract clauses and strictly output "*Insufficient data*" when queries fall outside verified records.

The automated test suite is housed in `tests/test_intelligence_agent.py` and can be triggered via the single-click PowerShell script `.\run_tests.ps1` or directly via `python -m pytest`.

2. Test Hierarchy & Pyramidal Strategy

The testing suite is structured into four distinct layers ensuring robust coverage from low-level functions to end-to-end integration:

Testing Tier	Target Module	Primary Objective	Test Isolation Method
Layer 1: Unit Tests	`ScheduleEngine`, `CostEngine`, `ProcurementEngine`, `ContractorEngine`	Verify arithmetic correctness for dates, float, burn rate, productivity deficit, and budget variance.	Isolated synthetic DataFrames with known mathematical ground truths.
Layer 2: Contract & RAG Tests	`ContractRAGEngine`	Verify TF-IDF vector indexing, relevance scoring thresholding, and anti-hallucination guards.	Structured clause datasets & verified negative (unsupported) queries.
Layer 3: Multi-Agent Logic	`ProjectRiskSupervisor`	Verify detection and resolution of inter-agent telemetry contradictions.	Synthetic conflicting agent payloads (e.g. on-time with cost overrun).
Layer 4: System Integration	`ProjectDataLoader`, `DataQualityChecker`, Streamlit App	Verify end-to-end ingestion across all 14 attached files and clean dashboard compilation.	Full production dataset (12,633 activities, 30 contracts, 15,571 dependencies).

3. In-Depth Breakdown of All 15 Acceptance Test Cases

`Test 1: test_date_parsing_and_missing_dates` UNIT TEST PASSED

Business Need: Construction schedules routinely contain incomplete activities where `actual_finish` is empty. A naive parser might crash, default missing dates to epoch zero, or incorrectly assume the activity took zero days.

Implementation: Injects an activity with `actual_finish: ""`. Tests that `pd.to_datetime()` does not mutate or crash, preserves `pd.isna(actual_finish)`, correctly classifies `is_in_progress == True`, and measures overdue days against the `cutoff_date`.

`Test 2 & 3: test_completed_activity_delay` UNIT TEST PASSED

Business Need: A completed activity must have its delay calculated strictly by comparing its actual completion date to its planned completion date, ignoring current reporting cutoff dates.

Implementation: Injects an activity with planned finish `2026-01-10` and actual finish `2026-01-18`. Asserts that `finish_delay_days == 8`, `delay_days == 8`, and `is_overdue == False` (because completed tasks are no longer actively overdue).

`Test 4: test_incomplete_overdue_activity` UNIT TEST PASSED

Business Need: For ongoing activities, project managers need to know how many days have elapsed past the baseline deadline as of the reporting cycle cutoff date.

Implementation: Injects an in-progress milestone activity with planned finish `2026-03-01` evaluated against a cutoff date of `2026-03-15`. Asserts that `is_overdue == True`, `overdue_days == 14`, and `milestone_delayed == True`.

`Test 5: test_zero_and_negative_delay` UNIT TEST PASSED

Business Need: The system must handle activities that finish ahead of schedule (negative delay) or exactly on target without producing false-positive delay alerts.

Implementation: Tests two activities: one finishing 4 days early (`finish_delay_days == -4`) and one finishing on the exact planned date (`finish_delay_days == 0`). Asserts that both are categorized as `ON_TIME` with zero penalty points.

Test 6: test_cost_variance_calculation

UNIT TEST | PASSED

Business Need: Project leadership requires accurate cost variance and daily burn rate tracking to calculate financial exposure on extended activities.

Implementation: Injects an activity with `baseline_cost = 50,000`, `actual_cost = 65,000`, and `delay_days = 5`. Asserts that `cost_variance == 15,000` (30% overrun) and calculates ongoing daily burn rate exposure.

Test 7: test_missing_budget_information

EDGE CASE | PASSED

Business Need: Unplanned site activities or emergency work orders may have a baseline budget of zero or null. Dividing by zero would crash the dashboard.

Implementation: Tests an activity with `baseline_cost = 0.0` and `actual_cost = 10,000.0`. Asserts that `cost_variance_pct` returns `0.0%` gracefully without raising a `ZeroDivisionError`.

Test 8: test_unmatched_procurement_records

EDGE CASE | PASSED

Business Need: Not all construction tasks require physical materials (e.g. surveying, site clearance, approvals). Unmatched activities must not trigger false inventory alerts.

Implementation: Queries procurement status for an activity with no matching BOQ rows. Asserts that `has_material_requirement == False`, `materials == []`, and `has_shortage == False`.

`Test 9: test_supplier_performance_calculation` UNIT TEST PASSED

Business Need: Procurement teams must distinguish vendor delivery shortages from quality inspection rejections.

Implementation: Supplies 1,000 received units, 900 accepted, and 100 rejected (10% defect rate) with warehouse reserved stock exceeding closing stock by 300 units. Asserts acceptance rate is exactly `90.0%` and stock deficit is `300.0` units.

`Test 10: test_rag_evidence_retrieval` RAG TEST PASSED

Business Need: Legal claims must be grounded in exact contractual language with verifiable section and clause identifiers.

Implementation: Indexes Clause 8.4 ("Delay and Liquidated Damages"). Queries with "*liquidated damages penalty delay*". Asserts top retrieved chunk is Clause 8.4, `penalty_applicable == True`, and cosine similarity exceeds relevance threshold.

`Test 11: test_unsupported_contract_questions` ANTI-HALLUCINATION PASSED

Business Need: LLM/RAG systems must never invent terms or penalties for out-of-scope inquiries.

Implementation: Queries the contract engine with an out-of-domain prompt: "*What is the software license agreement for the Salesforce CRM platform?*". Asserts the system returns `evidence_retrieved == False` and outputs "*Insufficient data*".

`Test 12: test_supervisor_conflict_resolution` AGENT TEST PASSED

Business Need: Specialist agents may report contradictory telemetry that must be detected and reconciled before executive reporting.

Implementation: Feeds the Supervisor an activity with `delay_days = 0` but a `cost_variance = 50,000`. Asserts the Supervisor flags a `Cost vs Schedule Discrepancy` and explains that the overrun is driven by unit rates rather than schedule delay.

`Test 13: test_local_fallback_without_api_credentials` OFFLINE TEST PASSED

Business Need: The core analytics and RAG capabilities must remain 100% operational offline without external API keys or cloud dependencies.

Implementation: Clears environment API keys, initializes the local TF-IDF retriever, and validates that question answering and clause retrieval function locally without network calls.

`Test 14: test_empty_datasets_handling` RESILIENCE TEST PASSED

Business Need: Corrupted or empty input files must fail gracefully rather than crashing the runtime server.

Implementation: Feeds empty DataFrames into `DataQualityChecker` and `ScheduleEngine`. Asserts that the audit score reflects missing data penalties and returns empty structured results safely.

`Test 15: test_integration_on_actual_workspace_data` INTEGRATION TEST PASSED

Business Need: Verify that the entire pipeline scales smoothly across the actual 100-project workspace dataset.

Implementation: Ingests the full 12,633-row activities dataset and 15,571 dependencies, executes `ScheduleEngine.analyze()`, and verifies KPI generation for project `PRJ-0001`.

`Test 16: test_dashboard_module_imports` SMOKE TEST PASSED

Business Need: Ensure that the Streamlit UI script has zero syntax errors, missing imports, or broken symbol links.

Implementation: Compiles `dashboard/app.py` into Python bytecode using `compile()` to verify syntactical integrity prior to web server launch.

4. Test Execution & Automation Architecture

The testing suite is configured for instant developer and CI/CD execution. The automated runner script `run_tests.ps1` configures the local `PYTHONPATH` and executes pytest with verbose output:

```
# Execution Command
$env:PYTHONPATH = "C:\Users\ompet\OneDrive\Desktop\infras_project\Infrastructure_Project_AI"
python -m pytest tests/test_intelligence_agent.py -v

# Verified Execution Output
============================= test session starts =============================
platform win32 -- Python 3.15.0, pytest-9.1.1, pluggy-1.7.0
cachedir: .pytest_cache
rootdir: C:\Users\ompet
plugins: anyio-4.15.1
collected 15 items

tests/test_intelligence_agent.py::test_date_parsing_and_missing_dates PASSED      [  6%]
tests/test_intelligence_agent.py::test_completed_activity_delay PASSED            [ 13%]
tests/test_intelligence_agent.py::test_incomplete_overdue_activity PASSED         [ 20%]
tests/test_intelligence_agent.py::test_zero_and_negative_delay PASSED             [ 26%]
tests/test_intelligence_agent.py::test_cost_variance_calculation PASSED           [ 33%]
tests/test_intelligence_agent.py::test_missing_budget_information PASSED          [ 40%]
tests/test_intelligence_agent.py::test_unmatched_procurement_records PASSED       [ 46%]
tests/test_intelligence_agent.py::test_supplier_performance_calculation PASSED    [ 53%]
tests/test_intelligence_agent.py::test_rag_evidence_retrieval PASSED              [ 60%]
tests/test_intelligence_agent.py::test_unsupported_contract_questions PASSED      [ 66%]
tests/test_intelligence_agent.py::test_supervisor_conflict_resolution PASSED      [ 73%]
tests/test_intelligence_agent.py::test_local_fallback_without_api_credentials PASSED [ 80%]
tests/test_intelligence_agent.py::test_empty_datasets_handling PASSED             [ 86%]
tests/test_intelligence_agent.py::test_integration_on_actual_workspace_data PASSED [ 93%]
tests/test_intelligence_agent.py::test_dashboard_module_imports PASSED            [100%]

============================= 15 passed in 2.90s ==============================
```

5. Data Ethics & Test Data Isolation

In compliance with strict data engineering standards, test isolation was maintained through clear boundaries:

- **No Mock Pollution:** Synthetic test data is isolated strictly inside individual unit test functions. It is never persisted into the `data/raw/` directory or merged with the actual 100-project dataset.
- **Zero In-Place File Mutation:** Ingestion and analytics tests operate on read-only in-memory copies of the raw CSVs, preserving original audit trails.
- **Zero Fabrication Rule:** Incomplete records return explicit `"Insufficient data"` status rather than fabricating optimistic metrics to make dashboards look artificially complete.

6. Key Interview Talking Points on Testing

How to Ace the "Testing" Question in Your Interview

- **"How did you test your RAG system?"** → *"We tested it symmetrically for precision and anti-hallucination. In Test 10, we validated that queries on liquidated damages retrieved Clause 8.4 with high cosine similarity. In Test 11, we passed out-of-domain queries and verified that the relevance floor tripped, returning 'Insufficient data' instead of guessing."*
- **"Why Pytest over manual verification?"** → *"Infrastructure data involves interdependent networks. A small code modification in float calculation could break downstream CPM propagation across 15,500 links. Pytest gives us 15 automated regression checks running in under 3 seconds on every commit."*
- **"How did you test multi-agent interactions?"** → *"In Test 12, we injected synthetic conflicting agent payloads—such as an activity having zero delay but a massive cost overrun—to prove that the Project Risk Supervisor detects the contradiction and reconciles it logically."*